

美团前端开发工程师（AICoding 方向）

一面（技术基础） · 1-6

1. 聊聊 JavaScript 的事件循环机制，宏任务和微任务的执行顺序是怎样的？
2. 介绍一下 CSS 中的 Flex 布局，如何实现一个垂直居中且两端对齐的容器？
3. 说一下 SSE（Server-Sent Events）和 WebSocket 的区别，为什么 AI 对话场景多用 SSE？
4. 手写一个深拷贝函数，需要考虑循环引用和 Symbol 类型。
5. 简单介绍一下 TypeScript 中的泛型，并写一个提取 Promise 返回类型的工具类型。
6. 给定一个字符串，请找出其中不含有重复字符的最长子串的长度。

二面（深入原理与项目） · 1-8

1. React 的 Fiber 架构解决了什么问题？它的工作原理是什么？
2. 在 AI 对话界面中，如何实现像 ChatGPT 那样的 Markdown 实时流式渲染？
3. 谈谈你对 Prompt Engineering 的理解，前端如何在代码中优化 Prompt 的质量？
4. 你们项目中是如何处理 AI 接口的长连接超时和异常重试的？
5. 什么是 RAG（检索增强生成）？前端在 RAG 流程中可以参与哪些工作？
6. 如何优化前端长列表的性能？如果列表项是 AI 生成的动态高度内容呢？
7. 谈谈前端安全，特别是 AI Coding 场景下如何防范 XSS 攻击？
8. 实现一个简单的 AI Agent 任务编排逻辑，前端如何处理多个 Tool Call 的串行或并行调用？

三面（架构与综合能力） · 1-5

1. 如果让你从零设计一个 Web IDE 中的 AI 代码助手插件，你会如何设计其整体架构？
2. 在 AI 响应速度（Latency）和生成质量（Quality）之间，你会采取哪些技术手段进行权衡和优化？
3. 谈谈 AI 对未来前端开发模式的影响，你认为前端工程师的核心竞争力会发生什么变化？
4. 聊聊你在过去一年中遇到的最具有挑战性的技术问题，你是如何定位并解决的？
5. 在团队中，如果 AI 生成的代码出现了线上故障，作为负责人你会如何制定 Code Review 规范来规避风险？

一面（技术基础）

1. 聊聊 JavaScript 的事件循环机制，宏任务和微任务的执行顺序是怎样的？

难度：★

考点：JS 异步执行机制

答案要点：

事件循环是 JS 处理异步操作的核心机制，通过任务队列的调度实现单线程非阻塞。

- 执行栈清空后，优先处理所有微任务队列中的任务
- 微任务处理完后，取出一个宏任务执行
- 常见的宏任务包括 setTimeout、setInterval、I/O
- 常见的微任务包括 Promise.then、MutationObserver、process.nextTick

常见坑：

- 误以为 await 后面的代码是宏任务，实际上 await 后的内容会被包装成微任务
- 忽略了 UI 渲染在事件循环中的插入时机

追问：

- 如果在微任务中递归产生新的微任务，会发生什么？
- Promise 的构造函数代码是同步还是异步执行的？

2. 介绍一下 CSS 中的 Flex 布局，如何实现一个垂直居中且两端对齐的容器？

难度：★

考点：CSS 布局基础

答案要点：

Flex 布局是基于轴线的弹性布局方案，通过设置容器属性来控制子元素的排列和空间分配。

- 设置 display: flex 开启布局
- 使用 align-items: center 实现交叉轴（垂直）居中
- 使用 justify-content: space-between 实现主轴两端对齐
- flex-grow、flex-shrink 和 flex-basis 的缩写含义

常见坑：

- 忘记设置容器高度导致垂直居中无效
- 混淆 justify-content 和 align-items 的作用对象

追问：

- flex: 1 代表什么意思？
- 当子元素宽度总和超过容器时，flex-shrink 是如何计算压缩比例的？

3. 说一下 SSE（Server-Sent Events）和 WebSocket 的区别，为什么 AI 对话场景多用 SSE？

难度：★★

考点：网络协议、AI 交互基础

答案要点：

SSE 是一种基于 HTTP 的单向推送协议，相比 WebSocket 更加轻量且原生支持断线重连。

- SSE 是单向的（服务端到客户端），WebSocket 是全双工双向通信
- SSE 基于标准 HTTP 协议，对防火墙和代理服务器更友好
- AI 场景下，模型生成是流式的且不需要客户端频繁反馈，SSE 协议开销更小
- SSE 支持自动重连机制，而 WebSocket 需要手动实现心跳和重连

常见坑：

- 认为 SSE 可以发送二进制数据（实际上主要传输文本）
- 忽略了浏览器对 HTTP/1.1 下 SSE 最大连接数的限制（通常为 6 个）

追问：

- 如何在前端处理 SSE 的流式数据解析？
- 如果需要客户端在对话过程中中断生成，SSE 该如何操作？

4. 手写一个深拷贝函数，需要考虑循环引用和 Symbol 类型。

难度：★★

考点：JS 基础、递归、复杂数据处理

答案要点：

深拷贝需要递归遍历对象的所有属性，并处理特殊类型以及避免循环引用导致的死循环。

- 使用 WeakMap 存储已拷贝的对象，解决循环引用问题
- 使用 Reflect.ownKeys 获取包括 Symbol 在内的所有键名
- 区分处理数组、对象、Date、RegExp 等特殊引用类型
- 递归终止条件是处理基本数据类型

常见坑：

- 使用 Map 而不是 WeakMap 导致内存无法被回收
- 忽略了 Symbol 类型的属性拷贝

代码示例：

代码块

```
1  function deepClone(obj, hash = new WeakMap()) {
2    if (obj === null || typeof obj !== 'object') return obj;
3    if (obj instanceof Date) return new Date(obj);
4    if (obj instanceof RegExp) return new RegExp(obj);
5    if (hash.has(obj)) return hash.get(obj);
6
7    const cloneObj = new obj.constructor();
8    hash.set(obj, cloneObj);
9
10   Reflect.ownKeys(obj).forEach(key => {
```

```
11     cloneObj[key] = deepClone(obj[key], hash);
12   });
13   return cloneObj;
14 }
```

5. 简单介绍一下 TypeScript 中的泛型，并写一个提取 Promise 返回类型的工具类型。

难度：★★

考点：TS 类型编程

答案要点：

泛型是指在定义函数、接口或类时不预先指定具体类型，而在使用时才指定类型的一种特性。

- 泛型提供了代码复用性并保留了类型信息
- 使用 extends 关键字进行类型约束
- 使用 infer 关键字在条件类型中进行类型推断
- 结合 Promise 结构进行模式匹配

常见坑：

- 无法处理嵌套 Promise 的情况
- 对 infer 的使用场景理解不深

代码示例：

代码块

```
1  type UnpackPromise<T> = T extends Promise<infer U> ? U : T;
2  // 进阶：递归处理
3  type DeepUnpackPromise<T> = T extends Promise<infer U> ? DeepUnpackPromise<U>
   : T;
```

6. 给定一个字符串，请找出其中不含有重复字符的最长子串的长度。

难度：★★

考点：算法、滑动窗口

答案要点：

通过滑动窗口的思想，维护一个不含重复字符的区间，动态更新最大长度。

- 使用 Map 或对象记录字符最后出现的位置
- 维护左指针 left 和右指针 i，当遇到重复字符时移动 left
- 每次循环计算当前窗口长度 i - left + 1 并更新最大值

- 时间复杂度要求为 $O(n)$

常见坑：

- 左指针 left 回退问题（应取当前位置和记录位置的最大值）
- 字符串为空或长度为 1 的边界处理

追问：

- 如果字符集包含 Unicode 字符，你的代码需要修改吗？
-

二面（深入原理与项目）

1. React 的 Fiber 架构解决了什么问题？它的工作原理是什么？

难度：★★★

考点：React 核心原理

答案要点：

Fiber 架构解决了 React 在处理大型组件树时，由于同步渲染阻塞主线程导致的掉帧问题。

- 引入了协作式调度，将渲染工作拆分为可中断的微小单元
- 分为 Reconciliation（协调）和 Commit（提交）两个阶段
- 协调阶段是可中断的，通过 requestIdleCallback 寻找执行时机
- 双缓存技术：在内存中构建 WorkInProgress Tree，完成后一次性切换

常见坑：

- 误认为 Fiber 提高了渲染速度（实际上是提高了响应速度）
- 混淆了 useEffect 和 useLayoutEffect 在 Fiber 周期中的执行时机

追问：

- 为什么 React 放弃了原有的递归 Diff，改用链表结构的 Fiber？

2. 在 AI 对话界面中，如何实现像 ChatGPT 那样的 Markdown 实时流式渲染？

难度：★★★

考点：流式渲染、性能优化、AI 交互

答案要点：

流式渲染需要结合 SSE 接收数据，并利用增量解析技术实时更新 DOM，同时处理代码高亮。

- 使用 markdown-it 或 marked 等解析库的增量更新策略
- 配合 highlight.js 或 prismjs 处理代码块，注意流式状态下的高亮闭合
- 优化渲染频率，避免每一个字符都触发一次重绘（如使用 requestAnimationFrame）
- 处理自动滚动到底部的逻辑，需判断用户是否正在向上浏览

常见坑：

- 每次收到新字符都全量渲染 Markdown 导致性能崩溃
- 代码块在未接收完时，标签未闭合导致的样式错乱

追问：

- 如何处理流式输出中的 LaTeX 数学公式渲染？

3. 谈谈你对 Prompt Engineering 的理解，前端如何在代码中优化 Prompt 的质量？

难度：★★

考点：AI 应用开发、Prompt 工程

答案要点：

Prompt Engineering 是通过精心设计的输入来引导模型产生高质量输出的技术，前端主要负责模板化和上下文管理。

- 采用 Few-shot（少样本）提示词，在 Prompt 中加入示例
- 结构化 Prompt：使用 XML 或 JSON 格式明确指令、背景、约束和输出格式
- 动态上下文注入：根据当前 UI 状态或用户操作历史组装 Prompt
- 引入 Chain of Thought（思维链），引导模型分步骤思考

常见坑：

- Prompt 过于冗长导致 Token 浪费和响应变慢
- 忽略了 Prompt 注入攻击（Prompt Injection）的风险

追问：

- 如何在前端实现 Prompt 的版本管理和 A/B 测试？

4. 你们项目中是如何处理 AI 接口的长连接超时和异常重试的？

难度：★★

考点：工程化、稳定性

答案要点：

针对 AI 接口响应慢、连接不稳定的特点，需要设计多层级的重试和容错机制。

- 实现指数退避算法（Exponential Backoff）的重试逻辑
- 区分业务错误（如 Token 超限）和网络错误，仅对网络错误重试
- 设置合理的超时时间，并提供前端的手动重试入口
- 使用 AbortController 实现请求的主动截断

常见坑：

- 无脑重试导致服务端压力激增
- 忽略了流式传输中断后的状态恢复逻辑

追问：

- 如果用户在生成过程中刷新页面，如何恢复之前的对话状态？

5. 什么是 RAG（检索增强生成）？前端在 RAG 流程中可以参与哪些工作？

难度：★★★

考点：AI 架构、RAG

答案要点：

RAG 通过从外部知识库检索相关信息并提供给大模型，解决了模型幻觉和时效性问题。

- 前端负责文档上传、解析预处理（如 PDF 分段预览）
- 实现知识库的管理界面，包括向量化进度的实时展示
- 在检索阶段，前端可以进行 Query 的预处理或多轮对话的意图压缩
- 渲染引用来源（Citations），点击来源跳转到文档具体位置

常见坑：

- 认为 RAG 只是简单的搜索加拼凑，忽略了召回质量对结果的影响
- 忽略了长文档在前端展示时的性能问题

追问：

- 如何在 UI 上优雅地展示模型引用的多个参考文档片段？

6. 如何优化前端长列表的性能？如果列表项是 AI 生成的动态高度内容呢？

难度：★★

考点：性能优化、虚拟列表

答案要点：

长列表优化通常采用虚拟列表技术，只渲染可视区域内的 DOM 节点。

- 动态高度方案：先给一个预估高度，渲染后获取真实高度并更新偏移量
- 使用 ResizeObserver 监听 AI 生成内容导致的元素高度变化
- 维护一个高度缓存表，记录每个索引对应的实际位置
- 滚动防抖与节流，结合 transform: translate3d 开启硬件加速

常见坑：

- 频繁触发重排导致的滚动卡顿
- 快速滚动时出现的白屏现象（需设置缓冲区 buffer）

追问：

- 虚拟列表在开启 Web 访问性（A11y）时会有什么问题？

7. 谈谈前端安全，特别是 AI Coding 场景下如何防范 XSS 攻击？

难度：★★

考点：Web 安全、AI 场景

答案要点：

AI 生成的内容具有不可预测性，必须经过严格的过滤和转义才能渲染到页面上。

- 对模型输出的 HTML 标签进行白名单过滤（使用 DOMPurify）
- 代码块预览功能应在沙箱环境（如 iframe 或 Web Worker）中运行
- 严格执行 CSP（内容安全策略），限制脚本的来源和执行
- 针对 AI 生成的脚本，禁止使用 eval 或 new Function 执行

常见坑：

- 认为 Markdown 解析器自带过滤功能（很多解析器默认不开启过滤）
- 忽略了 AI 可能诱导用户点击恶意链接的风险

追问：

- 如果 AI 生成了一个 React 组件，你如何安全地在页面上将其渲染出来？

8. 实现一个简单的 AI Agent 任务编排逻辑，前端如何处理多个 Tool Call 的串行或并行调用？

难度：★★★

考点：AI Agent、异步流控

答案要点：

Agent 模式下模型会返回函数调用指令，前端需要解析指令、执行本地工具并回传结果。

- 解析模型返回的 tool_calls 数组
- 根据依赖关系决定并行（Promise.all）或串行执行工具函数
- 维护一个任务状态机，实时向用户展示“正在查询天气”、“正在生成图表”等中间状态
- 错误处理：如果某个工具调用失败，需要将错误信息反馈给模型让其重试

常见坑：

- 忽略了工具调用可能产生的循环依赖
- UI 状态更新不及时导致用户以为程序卡死

追问：

- 如何在前端限制 Agent 的最大迭代次数以防止 Token 消耗失控？

三面（架构与综合能力）

1. 如果让你从零设计一个 Web IDE 中的 AI 代码助手插件，你会如何设计其整体架构？

难度：★★★

考点：系统设计、架构思维

答案要点：

一个成熟的 AI 代码助手需要考虑编辑器集成、上下文感知、模型通信和用户反馈闭环。

- 表现层：基于 Monaco Editor 或 VS Code API 的装饰器、补全面板、侧边栏对话
- 核心层：上下文提取引擎（获取当前文件、关联文件、Git 信息）、Prompt 编排器
- 通信层：支持 SSE 的流式请求封装，具备请求合并与取消机制
- 策略层：缓存策略（LRU 存储常用补全）、预取策略（光标停顿时预生成）
- 监控层：埋点记录采纳率、生成耗时、Token 消耗等关键指标

常见坑：

- 忽视了代码隐私安全，未对敏感信息进行脱敏
- 补全触发频率过高导致严重的性能损耗和成本浪费

追问：

- 如何实现“根据当前报错自动修复代码”的功能？

2. 在 AI 响应速度（Latency）和生成质量（Quality）之间，你会采取哪些技术手段进行权衡和优化？

难度：★★★

考点：性能优化、用户体验

答案要点：

通过分级模型策略、流式展示和感知优化来平衡响应速度与质量。

- 模型分级：简单任务（如代码格式化）用小模型，复杂逻辑用大模型
- 投机采样（Speculative Decoding）思想的前端应用：预加载常见回复
- 交互优化：流式输出降低首字响应时间（TTFT），配合 Loading 动画缓解焦虑
- 语义缓存：对相似的提问直接命中缓存，不请求模型

常见坑：

- 过度追求质量导致首屏等待时间超过 10 秒，用户流失
- 缓存策略失效导致陈旧信息误导用户

追问：

- 如何量化评价 AI 助手的“生成质量”？

3. 谈谈 AI 对未来前端开发模式的影响，你认为前端工程师的核心竞争力会发生什么变化？

难度：★★

考点：行业视野、职业思考

答案要点：

AI 将从“提效工具”演变为“开发伙伴”，前端工作重心将从手写代码转向系统设计和 AI 编排。

- 提效：重复性的 UI 编写、单元测试生成、文档撰写将被 AI 取代
- 转型：前端需要掌握模型调用、Prompt 调优、RAG 落地等 AI 相关技能
- 核心竞争力：复杂业务逻辑的抽象能力、极致的用户体验把控、对 AI 产出内容的 Review 和纠错能力
- 领域扩展：前端将更多参与到模型后处理、Agent 流程设计等后端边界工作

常见坑：

- 过于悲观认为前端会消失，或过于乐观认为 AI 没有任何门槛

追问：

- 你最近在关注 AI 领域的哪些新技术或新论文？

4. 聊聊你在过去一年中遇到的最具有挑战性的技术问题，你是如何定位并解决的？

难度：★★

考点：问题解决能力、复盘习惯

答案要点：

这是一个开放性问题，重点考察解决问题的思路、使用的工具以及最后的沉淀。

- 背景：清晰描述业务场景和技术栈
- 冲突：遇到了什么超乎寻常的 Bug 或性能瓶颈（如 AI 场景下的内存泄漏）
- 行动：使用了哪些调试工具（Chrome DevTools, Performance 采样, 抓包分析），尝试了哪些方案
- 结果：最终的解决方案是什么，带来了多少性能提升或业务价值
- 反思：如何避免同类问题再次发生，是否沉淀了通用组件或文档

常见坑：

- 描述过于琐碎，没有体现技术深度
- 解决方案完全依赖他人，没有自己的思考过程

5. 在团队中，如果 AI 生成的代码出现了线上故障，作为负责人你会如何制定 Code Review 规范来规避风险？

难度：★★

考点：团队协作、质量保障

答案要点：

AI 生成的代码必须遵循“信任但要核实”的原则，建立专门的 AI 代码准入机制。

- 强制标注：所有由 AI 生成的代码块必须有特殊注释标注
- 单元测试覆盖：AI 生成的代码必须配有同步生成的单元测试且通过
- 重点审查：关注边界条件、安全漏洞（如 SQL 注入）和高性能损耗逻辑

- 灰度发布：AI 驱动的功能必须经过严格的灰度观察期

常见坑：

- 盲目信任 AI 产出，跳过人工审核
- 规范过于繁琐导致开发效率反而下降

追问：

- 如何利用 AI 本身来辅助进行 Code Review？